



ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Faculty of Electrical and Computer Engineering

“Acceptance Testing”

WORKSHOP - FIRST TERM

SOFTWARE ENGINEERING II

Group Members:

Adrián Fierro José Andrés
Romero Yagual Bruno Rafael
Otero Limones Alex David
Pazmiño Reyna Jaren Aaron
Robinson Feraud Steve William

Guayaquil-Ecuador

2026 - 2027

1. Introduction

Behavior-Driven Development (BDD) is a methodology used in modern software engineering, bridging the communication gap between business stakeholders and technical teams. By fostering cross-role collaboration, BDD establishes a shared understanding of system problems and produces living documentation that is continuously verified against actual system behavior. A core component of this methodology is Acceptance Testing, a formal evaluation level conducted to determine whether a system satisfies established acceptance criteria and meets user requirements for delivery.

This practice report details the development and validation of a command-line Inventory Manager application using Python and the Behave testing framework. The application implements the core logic for fundamental product management tasks, specifically adding, listing, updating, and removing items, validated through executable acceptance tests. The objectives of this workshop are to implement an inventory logic and validate the system functionality through executable Gherkin syntax scenarios. Through this iterative process, the project demonstrates how automated acceptance criteria seamlessly guide software quality and functional alignment.

2. Development

Tool Configuration

First, we set up a virtual environment (venv) for Python:

Windows

```
python -m venv venv  
venv\Scripts\activate
```

macOS / Linux

```
python3 -m venv venv  
source venv/bin/activate
```

Next, we installed the `behave` library so that it would be available within our environment and could be used in the project.

```
pip install behave
```

Once this is done, we will proceed to write the corresponding code.

Requirements and Code Creation

Requirements & Quality Objectives

The purpose of this application is to implement a command-line InventoryManager system capable of product tracking. Per core specifications, each item within the system tracks four minimum required attributes: product_id, name, quantity, and price. The core codebase provides capability to add, verify, list, get a combo (a transaction with more than one product) and safely delete entries by their name keys.

The following table shows the implemented features:

Member	Feature
José Adrián	Add a product to the inventory
Jaren Pazmiño	List all products in the inventory
Alex Otero	Remove a product from the inventory
Steve Robinson	Update the quantity of a product
Bruno Romero	Combo Management

Core Source Code

inventory.py

```
class Product:
    def __init__(self, product_id, name, quantity, price):
        self.id = product_id
        self.name = name
        self.quantity = quantity
        self.price = price

class InventoryManager:
    def __init__(self):
        self.products = {}
        self.combos = {}

    def add_product(self, product_id, name, quantity, price):
        if name in self.products:
            return False
        self.products[name] = Product(product_id, name, quantity, price)
        return True

    def has_product(self, name):
        return name in self.products
```

```
def list_products(self):
    return list(self.products.keys())

def update_quantity(self, name, quantity):
    if name in self.products:
        self.products[name].quantity = quantity
        return True
    return False

def remove_product(self, name):
    if name in self.products:
        del self.products[name]
        return True
    return False

def create_combo(self, combo_name, product_names):
    for name in product_names:
        if not self.has_product(name):
            return False

    self.combos[combo_name] = product_names
    return True

def has_combo(self, combo_name):
    return combo_name in self.combos

def get_combo_products(self, combo_name):
    return self.combos.get(combo_name, [])
```

Feature File Creation

The BDD feature file has been created under `features/inventory.feature`. This specifications suite maps out business rules written in natural Gherkin syntax.

`inventory.feature`

```
#language: en
```

```
Feature: Inventory Management
```

```
Scenario: Add a product to the inventory
```

```
Given the inventory is empty
```

```
When the user adds a product "Coffee"
```

```
Then the inventory should contain "Coffee"
```

```
Scenario: List all products in the inventory
```

```
Given the inventory contains a product "Coffee"
```

```
And the inventory contains a product "Tea"
```

```
When the user lists all products
```

```
Then the inventory should show "Coffee" and "Tea"
```

```
Scenario: Update the quantity of a product
```

```
Given the inventory contains products:
```

```
| Product | Quantity | Id | Price |
```

```
| Coffee | 10 | PROD-67 | 2.5 |
```

```
When the user updates product "Coffee" to quantity "25"
```

```
Then the inventory should show product "Coffee" with quantity  
"25"
```

```
Scenario: Remove a product from the inventory
```

```
Given the inventory contains a product "Coffee"
```

```
When the user removes the product "Coffee"
```

```
Then the inventory should not contain "Coffee"
```

```
Scenario: Create a composite product combo
```

```
Given the inventory contains a product "Coffee"
```

```
And the inventory contains a product "Sugar"
```

```
When the user creates a combo "Breakfast Pack" with "Coffee,  
Sugar"
```

```
Then the inventory should contain the combo "Breakfast Pack"  
with "Coffee, Sugar"
```

Acceptance Tests Creation

The automated step testing layout is stored inside `features/steps/inventory_steps.py`. State sharing is isolated cleanly across steps utilizing Behave's structured context hook lifecycle rather than global memory stores.

`inventory_steps.py`

```
from behave import given, when, then
from inventory import InventoryManager

@given('the inventory is empty')
def step_impl_inventory_is_empty(context):
    context.inventory = InventoryManager()

@given('the inventory contains a product "{product_name}"')
def step_impl_inventory_contains_product(context, product_name):
    if not hasattr(context, 'inventory'):
        context.inventory = InventoryManager()

    product_id = f'PROD-{len(context.inventory.products) + 1:03d}'
    quantity = 10
    price = 5.0
    context.inventory.add_product(product_id, product_name,
    quantity, price)

@when('the user adds a product "{product_name}"')
def step_impl_add_product(context, product_name):
    product_id = 'PROD-001'
    quantity = 10
    price = 5.0
    context.inventory.add_product(product_id, product_name,
    quantity, price)

@when('the user lists all products')
def step_impl_list_all_products(context):
    context.product_names = context.inventory.list_products()

@then('the inventory should contain "{product_name}"')
def step_impl_inventory_contains(context, product_name):
    assert context.inventory.has_product(product_name), f'Product
    "{product_name}" not found in the inventory'

@then('the inventory should show "{first_product}" and
    "{second_product}"')
def step_impl_inventory_should_show_products(context,
    first_product, second_product):
```

```

        expected_products = [first_product, second_product]
        assert context.product_names == expected_products, (
            f'Expected products {expected_products}, but got
{context.product_names}'
        )

@given('the inventory contains products:')
def step_impl(context):
    context.inventory = InventoryManager()
    for row in context.table:
        context.inventory.add_product(row['Id'], row["Product"],
int(row["Quantity"]), float(row["Price"]))

@when('the user updates product "{product_name}" to quantity
"{product_quantity}"')
def step_impl(context, product_name, product_quantity):
    context.inventory.update_quantity(product_name,
int(product_quantity))

@then('the inventory should show product "{product_name}" with
quantity "{product_quantity}"')
def step_impl(context, product_name, product_quantity):
    assert context.inventory.products[product_name].quantity ==
int(product_quantity), f'Product "{product_name}" has not the
expected quantity'

@when('the user removes the product "{product_name}"')
def step_impl_remove_product(context, product_name):
    context.inventory.remove_product(product_name)

@then('the inventory should not contain "{product_name}"')
def step_impl_inventory_should_not_contain(context, product_name):
    assert not context.inventory.has_product(product_name),
f'Product "{product_name}" was found in the inventory, but it
should have been removed.'

@when('the user creates a combo "{combo_name}" with
"{products_str}"')
def step_impl_create_combo(context, combo_name, products_str):
    product_names = [p.strip() for p in products_str.split(',')]
    context.inventory.create_combo(combo_name, product_names)

@then('the inventory should contain the combo "{combo_name}" with
"{products_str}"')
def step_impl_verify_combo(context, combo_name, products_str):
    expected_products = [p.strip() for p in
products_str.split(',')]

```



```
    assert context.inventory.has_combo(combo_name), f'Combo  
"{combo_name}" was not found in the inventory.'  
  
    actual_products =  
context.inventory.get_combo_products(combo_name)  
    assert actual_products == expected_products, f'Expected  
{expected_products} but got {actual_products}'
```

Test Run Evidence

The output log below confirms local framework execution via terminal calls

```
(venv) PS C:\Users\alexo\Documents\Projects\T7-AcceptanceTesting> behave
USING RUNNER: behave.runner:Runner
Feature: Inventory Management # features/inventory.feature:3

  Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:34 0.000s
  Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:34
  Then the inventory should show "Coffee" and "Tea" # features/steps/inventory_steps.py:39 0.000s
  Then the inventory should show "Coffee" and "Tea" # features/steps/inventory_steps.py:39
  Given the inventory contains products: # features/steps/inventory_steps.py:46 0.000s
    | Product | Quantity | Id | Price | # features/steps/inventory_steps.py:46
  Then the inventory should show product "Coffee" with quantity "25" # features/steps/inventory_steps.py:56 0.000s
  Then the inventory should show product "Coffee" with quantity "25" # features/steps/inventory_steps.py:56
  Then the inventory should not contain "Coffee" # features/steps/inventory_steps.py:65 0.000s
  Then the inventory should not contain "Coffee" # features/steps/inventory_steps.py:65
  Then the inventory should contain the combo "Breakfast Pack" with "Coffee, Sugar" # features/steps/inventory_steps.py:75 0.000s
  Then the inventory should contain the combo "Breakfast Pack" with "Coffee, Sugar" # features/steps/inventory_steps.py:75
1 feature passed, 0 failed, 0 skipped
5 scenarios passed, 0 failed, 0 skipped
17 steps passed, 0 failed, 0 skipped
Took 0min 0.009s
(venv) PS C:\Users\alexo\Documents\Projects\T7-AcceptanceTesting>
```

Tests/Code Correction Evidence

By restructuring the step definitions to use Behave's native context lifecycle instead of global variables, every test scenario runs inside a completely isolated environment. This clean separation, combined with replacing hardcoded values with dynamic ID generation, prevents data leaks and ID collisions between independent scenarios. During the iterative review, the suite was re-run and no failing tests were found: the final execution log confirms a flawless test suite health status (5 scenarios passed, 0 failed, 0 skipped), validating all core inventory operations, that is, adding, listing, updating, and removing products, as well as creating a composite product combo.

3. Link to Repository

<https://github.com/Arzel01/T7-AcceptanceTesting>

4. Conclusions and recommendations

Conclusions

The successful implementation of the command-line Inventory Manager application demonstrated that Behavior-Driven Development (BDD) effectively bridges the gap between business requirements and technical execution by converting formal acceptance criteria into executable code. Through the use of the Behave framework, the system successfully validated core inventory functionalities, such as adding, listing, updating, and removing products based on specific user needs. Ultimately, following the workshop's structured three-part iterative process, comprising project setup, test development, and iterative review, allowed for the systematic refinement and validation of the implementation, resulting in a robust, functional deliverable that completely fulfills all established acceptance criteria.

Recommendations

- Utilize Behave's Context Object for State Management: It is highly recommended to manage application state using Behave's native context object rather than relying on global variables. Because the context object isolates data execution parameters, it ensures that Behave provides a fresh environment for each individual test case, successfully preventing data leaks between distinct testing scenarios.
- Utilize Scenario Outlines and Data Tables for Scale: When dealing with multi-attribute product profiles or scaling your test scenarios, leverage structured data formats like context.table and Gherkin's Scenario Outline. Using these tools provides an efficient, clean approach to parsing tabular inputs, allowing you to easily loop through rows and verify multiple product attributes simultaneously by their column headers.
- Implement Comprehensive Failed Interaction Scenarios: It is highly recommended to explicitly write scenarios that check what happens when something goes wrong, such as searching for or removing a product that is not present in the inventory. These failure-path tests should utilize targeted assertions to verify that the application returns the correct error message and that the underlying system state remains unchanged.

5. References

- [1] Software Testing Fundamentals, “Acceptance Testing,” [Online]. Available: <http://softwaretestingfundamentals.com/acceptance-testing/>.
- [2] R. Hewitt, “Gherkin for Business Analysts,” Modern analyst, 2017. [Online]. Available: <https://www.modernanalyst.com/Resources/Articles/tabid/115/ID/3810/Gherkin-for-Business-Analysts.aspx>. [Accessed 14 Marzo 2020].
- [3] “Gherkin Reference,” [Online]. Available: <https://cucumber.io/docs/gherkin/reference/>. [Accessed Marzo 2020].
- [4] Behave, “Read the Docs,” [Online]. Available: <https://behave.readthedocs.io/en/latest/tutorial/#feature-files>.
- [5] B. A. Reference. [Online]. Available: <https://behave.readthedocs.io/en/latest/api/>.
- [6] S. Vergara, “¿Qué es BDD (Behavior Driven Development)?,” ITDO, 18 Julio 2019. [Online]. Available: <https://www.itdo.com/blog/que-es-bdd-behavior-driven-development/>. [Accessed 14 Marzo 2020].